Raft Implementation with Spring Boot

A Spring Boot application implementing the Raft consensus algorithm to manage a cluster of nodes. This project ensures consistency and reliability through leader election and state replication. **This is a proof of concept designed to demonstrate how the Raft algorithm operates.**

Use Docker Compose to start a Raft cluster with three nodes.

1. **Start the Containers:**

   ```
   docker compose up -d
   ```

2. **Verify the Containers:**

   Check the logs to ensure each node starts correctly.

   ```
   docker logs -f node1
   docker logs -f node2
   docker logs -f node3
   ```

Node State Management

Each node in the Raft cluster can be in one of the following states:

- **Follower:** Passive state, waiting for instructions from the leader.
- **Candidate:** State entered when a node attempts to become a leader.
- **Leader:** The node responsible for managing the cluster and coordinating operations.
- **Down:** The node is not active or has failed.

The state of a node is persisted and can be retrieved using the `/raft/status` endpoint.

Monitoring

To monitor the status of all nodes in the Raft cluster:

1. Navigate to the `/monitor` endpoint of any node. For example, if running on port `8000` :

   ```
   http://localhost:8000/monitor
   ```

   Raft Monitoring

2. The monitoring page displays the status of all nodes, including their current state and term.
3. Only for debug purposes, in the page `/monitor` you can `stop` / `resume` a node, this will simulate a node failure and permit to see the behavior of the cluster.

   Raft Monitoring

API Endpoints

- **Start Election**
  - **Endpoint:** `POST /raft/start-election`
  - **Description:** Initiates a new election in the Raft cluster.
- **Request Vote**
  - **Endpoint:** `POST /raft/request-vote`
  - **Description:** Handles vote requests from candidate nodes.
  - **Request Body:**

    ```
    {
      "candidateId": "node2",
      "candidateTerm": 2
    }
    ```

- **Initialize Node**
  - **Endpoint:** `POST /raft/initialize`
  - **Description:** Initializes the node within the Raft cluster.
- **Receive Heartbeat**
  - **Endpoint:** `POST /raft/heartbeat`
  - **Description:** Receives a heartbeat signal from the leader node.
- **Get Node Status**
  - **Endpoint:** `GET /raft/status`
  - **Description:** Retrieves the current status of the node.
- **Stream Nodes Status**
  - **Endpoint:** `GET /raft/status-stream`
  - **Description:** Streams the status of all nodes using Server-Sent Events (SSE).

(Only for debug purposes)

- **Stop Node**
  - **Endpoint:** `GET /raft/stop`
  - **Description:** Stop the node
- **Resume Node**
  - **Endpoint:** `GET /raft/resume`
  - **Description:** Resume the node